



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Spring, Year: 2025), B.Sc. in CSE (Day)

Lab Report: 05

Course Title: Data Mining Lab
Course Code: CSE 436
Section: 212_D2

Lab Manual 09 (updated)

Implementation of DBSCAN Clustering Algorithm

Student Details

Name	ID
Md. Almahmud	212002142

Submission Date : 30-04-2025
Course Teacher's Name : Md. Atikuzzaman

Lab Report Status

Marks :Signature.....

Comments :Date :.....

1. TITLE OF THE LAB EXPERIMENT:

Implementation of DBSCAN (Density-Based Spatial Clustering of Applications with Noise) Clustering Algorithm Using Python. Use “*Wholesale customers data.csv*” Dataset.

2. OBJECTIVES/AIM:

- To implement the DBSCAN algorithm in Python for identifying clusters of customers based on their purchasing behavior.

3. MOTIVATION:

- DBSCAN helps find groups of customers who spend in similar ways, even if those groups are not shaped normally or clearly separated.
- It also points out customers who spend very differently from others, which can be useful to spot special cases or unique buying habits.

4. PROCEDURE:

- Load the dataset “*Wholesale customers data.csv*”.
- Perform preprocessing: handling missing values (if any), normalizing or scaling data.
- Apply the DBSCAN algorithm with appropriate parameters (epsilon and minimum samples).
- Analyze resulting clusters and identify noise points.
- Visualize the clusters using suitable plotting techniques (e.g., scatter plots with color-coded clusters).

5. CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA

file_path = '/content/Wholesale customers data.csv'
df = pd.read_csv(file_path)
df.head()
df_cleaned = df.drop(columns=['Channel', 'Region'], errors='ignore')
df_cleaned.head()
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_cleaned)
dbscan = DBSCAN(eps=1.9, min_samples=4)
clusters = dbscan.fit_predict(scaled_data)
df['Cluster'] = clusters
print(df)
pca = PCA(n_components=2)
pca_result = pca.fit_transform(scaled_data)
df['PCA1'] = pca_result[:, 0]
df['PCA2'] = pca_result[:, 1]
```

```

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PCA1', y='PCA2', hue='Cluster',
palette='Set2', s=100)
plt.title('DBSCAN Clustering Visualization (PCA Reduced)')
plt.legend(title='Cluster')
plt.show()

print("\n Visualizing the Clusters.")
plt.figure(figsize=(8, 6))
unique_labels = set(clusters)
colors = plt.cm.get_cmap("tab10", len(unique_labels))

for label in unique_labels:
    if label == -1:
        cluster_points = df_cleaned[clusters == label]
        # Access columns by name using .loc for location-based
        indexing
        plt.scatter(cluster_points.loc[:, df_cleaned.columns[0]],
cluster_points.loc[:, df_cleaned.columns[1]],
                    label='Noise', color='black', edgecolor='k')
    else:
        cluster_points = df_cleaned[clusters == label]
        # Access columns by name using .loc for location-based
        indexing
        plt.scatter(cluster_points.loc[:, df_cleaned.columns[0]],
cluster_points.loc[:, df_cleaned.columns[1]],
                    label=f'Cluster {label}', edgecolor='k')

plt.title("DBSCAN Clustering")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
plt.show()

print("\nCluster Value Counts:")
print(df['Cluster'].value_counts())

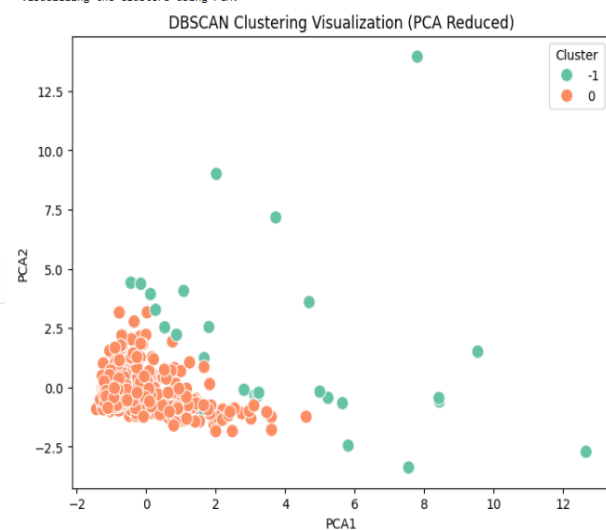
```

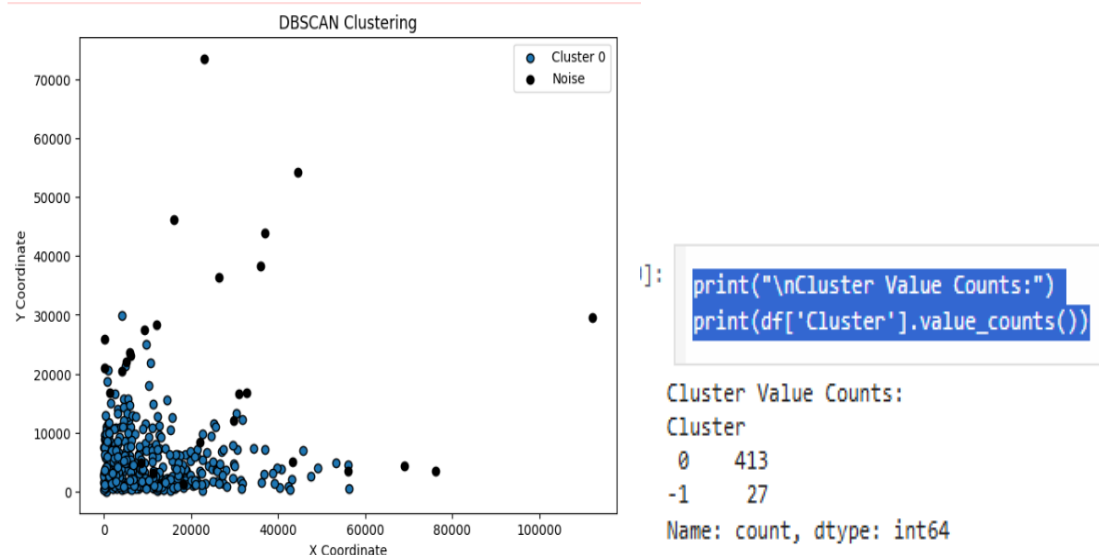
6. OUTPUT:

df_cleaned.head()

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	12669	9656	7561	214	2674	1338
1	7057	9810	9568	1762	3293	1776
2	6353	8808	7684	2405	3516	7844
3	13265	1196	4221	6404	507	1788
4	22615	5410	7198	3915	1777	5185

Visualizing the Clusters using PCA.





7. DISCUSSION:

The dataset contains customer spending details across different product categories such as fresh items, milk, grocery, frozen products, detergents, and delicatessen. Before clustering, the data was preprocessed and normalized to make all features comparable. Then, the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm was applied to group customers based on similar spending patterns. Unlike other clustering methods, DBSCAN can also detect outliers—customers with unusual or rare purchase behavior—labeling them as noise.

Choosing the right values for DBSCAN's parameters—epsilon (eps) and minimum samples (min_samples)—was very important. A common way to find these is by using the k-distance graph, which helps decide the best distance threshold. The results showed that DBSCAN is especially good for identifying clusters of uneven shapes and works well with irregularly distributed data. It also handled noise better than K-Means, making it a strong choice when analyzing complex customer spending behavior.